

# Phase 3: Anforderungsanalyse bis Lösung und Lessons Learned

## Projekt Cloud Programming

Jonas Habel  
Matrikelnummer: 9227109

18. Februar 2026

### Zusammenfassung

Diese Ausarbeitung beschreibt den vollständigen Weg der Projektidee von der Anforderungsanalyse über die methodische Auswahl der Plattform bis zur technischen Lösung in Azure mit Terraform. Der Fokus liegt auf fachlicher Problemabgrenzung, einer kritischen Architekturentscheidung und reproduzierbaren Deployment-Schritten ohne Screenshots. Zusätzlich werden zwei in Phase 3 behobene Issues systematisch dokumentiert: ein Terraform-Validierungsfehler in `variables.tf` sowie die Ergänzung sicherheitsrelevanter HTTP-Header in der NGINX-Konfiguration trotz bereits erzwungenem HTTPS in Front Door.

## 1 Einführung: Anforderungsanalyse und Ziele

### 1.1 Ausgangssituation und Problemabgrenzung

Ausgangspunkt des Projekts ist die Bereitstellung einer Unternehmenswebsite in einer Cloud-Umgebung. Die eigentliche Webanwendung ist bewusst einfach gehalten (statische HTML-Seite), damit die Bewertung auf dem Architektur- und Betriebsanteil liegt. Die technische Problemstellung liegt damit nicht in komplexer Fachlogik, sondern in den nicht-funktionalen Anforderungen:

- hohe Verfügbarkeit auch bei regionalen Störungen,
- weltweit niedrige Latenz für Nutzende,
- automatische Skalierung bei Lastspitzen,
- sichere und reproduzierbare Bereitstellung per Infrastructure as Code (IaC).

Die Problemabgrenzung ist klar: Es geht nicht um die Entwicklung eines Feature-reichen Frontends oder Backends, sondern um den Nachweis, dass ein cloud-natives Bereitstellungsmodell mit Multi-Region-Betrieb, globalem Routing und standardisierter Automatisierung sauber implementiert werden kann.

### 1.2 Zielsetzung

Die Zielsetzung wurde in technische Teilziele übersetzt:

1. **Verfügbarkeit:** Betrieb der Anwendung in zwei Azure-Regionen.
2. **Globale Erreichbarkeit:** ein einheitlicher globaler Entry Point mit intelligentem Routing.
3. **Skalierbarkeit:** automatische Anpassung der Replikazahl an HTTP-Last.
4. **Sicherheit:** HTTPS-End-to-End-Routing sowie harte Sicherheitsheader auf Webserver-Ebene.
5. **Reproduzierbarkeit:** vollständige Infrastrukturdefinition in Terraform und wiederholbares Deployment per Skript.

## 2 Methodik: Plattformauswahl, Vergleich und Architekturansatz

### 2.1 Vorgehensmodell

Die Methodik folgt einem iterativen, risikoorientierten Ablauf: zuerst Anforderungen präzisieren, dann Dienste vergleichen, danach Architektur implementieren und schließlich das Ergebnis gegen die Zielanforderungen prüfen. Die in Phase 2 entstandene Prozessdarstellung wird in Phase 3b weiterverwendet:

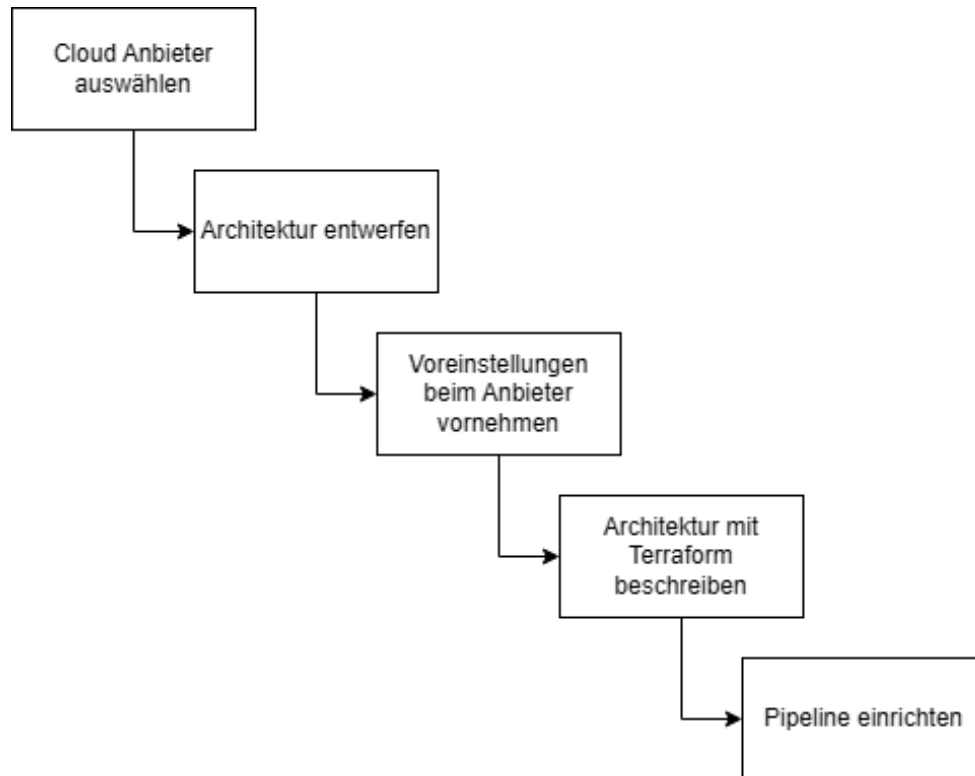


Abbildung 1: Vorgehensmodell aus Phase 2

Die praktische Umsetzung war dabei bewusst als Schleife organisiert und nicht als einmaliger “Big-Bang”-Deploy. Nach jedem technischen Schritt wurde geprüft, ob das Ergebnis noch zur Zielsetzung passt: Nach Infrastrukturänderungen folgte `terraform validate`, bei Containeränderungen ein erneuter Build/Push-Zyklus, und bei Routing-Anpassungen eine Erreichbarkeits- und Failover-Prüfung über den Front-Door-Endpunkt. Dieser zyklische Ansatz hat zwei Vorteile gebracht: Fehler wurden früh erkannt, und die Auswirkung einzelner Änderungen blieb klar nachvollziehbar.

### 2.2 Explizite Entscheidungskriterien

Die Auswahl der Cloud-Dienste erfolgte explizit entlang messbarer Kriterien:

- **Hohe Verfügbarkeit:** aktive Bereitstellung in mindestens zwei Regionen.
- **Automatische Skalierung:** native Lastskalierung ohne manuelles Node-Management.
- **Sicherheit:** TLS-Absicherung, least-privilege Zugriff und Härtung von HTTP-Responses.
- **Betriebsaufwand:** möglichst geringer Plattform-Overhead (managed Services statt Self-Managed Cluster).
- **Automatisierbarkeit:** IaC- und CI/CD-Fähigkeit mit stabilen Providern und klaren APIs.
- **Kosten- und Komplexitätsprofil:** angemessen für einen kleinen, aber realistischen Demonstrator.

## 2.3 Plattformvergleich und kritische Beurteilung

Fachlich wären mehrere Plattformkombinationen möglich gewesen. Die Entscheidung wurde nicht über ein einzelnes Kriterium getroffen, sondern als Trade-off:

Tabelle 1: Vergleich möglicher Plattformoptionen

| Kriterium        | Azure (gewählt)                                    | AWS (Alternative)                             | GCP (Alternative)                        |
|------------------|----------------------------------------------------|-----------------------------------------------|------------------------------------------|
| Globales Routing | Front Door mit Any-cast, Health Probe und Failover | CloudFront + Route 53 + ALB                   | Cloud CDN + Global LB                    |
| Containerbetrieb | Container Apps (serverless, autoscaling)           | ECS/Fargate oder EKS                          | Cloud Run oder GKE                       |
| IaC-Reife        | Terraform AzureRM gut etabliert                    | Terraform AWS sehr ausgereift                 | Terraform Google ausgereift              |
| Betriebsaufwand  | gering bis mittel (managed)                        | mittel (je nach ECS/EKS Wahl)                 | gering bis mittel                        |
| Projektfitt      | sehr gut: direkte Kombination Front Door + ACA     | gut, aber mehr Integrationsaufwand im Entwurf | gut, aber geringere Vorerfahrung im Team |

**Kritische Bewertung:** Die Wahl von Azure ist für diese Problemstellung sinnvoll, weil Front Door und Container Apps die Kernanforderungen direkt adressieren. Die Alternative *AKS + Ingress + eigener globaler Traffic-Manager* wäre flexibler, hätte jedoch den Betriebsaufwand unverhältnismäßig erhöht. Ebenfalls möglich wäre eine *Static-Web-App-Lösung* gewesen; diese wäre für reinen HTML-Content einfacher, bietet aber weniger Lernwert für containerisierte Multi-Region-Deployments und spätere Backend-Erweiterungen.

## 2.4 Cloud-Architektur als Zielbild

Das frühe Architekturziel aus Phase 1 wurde in Phase 2/3 konkretisiert. Die folgenden Diagramme zeigen die Entwicklung vom Konzept zur umgesetzten Struktur:

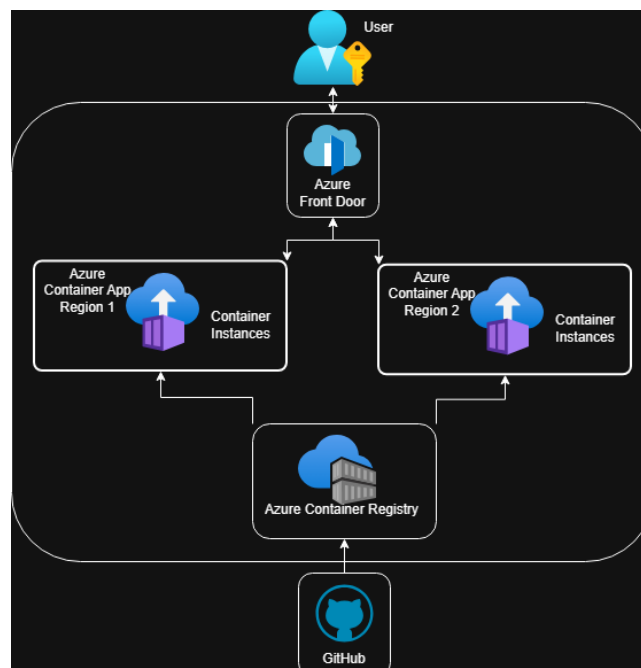


Abbildung 2: Kompakte Umsetzungsarchitektur

## 3 Lösung: Technische Umsetzung und IaC-Struktur

### 3.1 Architektur der implementierten Lösung

Die finale Lösung besteht aus folgenden Schichten:

1. **Artefakt-Schicht:** Docker-Image der statischen Web-App.
2. **Registry-Schicht:** Azure Container Registry (ACR) für Image-Hosting.
3. **Compute-Schicht:** zwei Azure Container Apps (Primary/Secondary) in verschiedenen Regionen.
4. **Observability-Schicht:** je Region ein Log-Analytics-Workspace und ein Container-App-Environment.
5. **Edge-Schicht:** Azure Front Door als globaler Einstiegspunkt mit HTTPS-Weiterleitung und Health-basiertem Routing.
6. **Identity-Schicht:** User Assigned Managed Identity und AcrPull-Rolle für Registry-Zugriffe.

Der Request-Fluss ist klar definiert: Client → Front Door Endpoint → gesundes Origin (primär, ansonsten sekundär) → Container App Ingress.

### 3.2 IaC-Struktur über “Module” und Variablen

Auch ohne ausgelagerte Terraform-Unterordner ist die Infrastruktur fachlich modular organisiert. Die Dateigruppierung im Ordner `infra/` bildet logische Module:

Tabelle 2: Logische Terraform-Module nach Verantwortungsbereich

| Bereich             | Dateien                                                                                                              | Aufgabe                                        |
|---------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Core                | <code>versions.tf</code> ,<br><code>resource-group.tf</code> ,<br><code>locals.tf</code> , <code>variables.tf</code> | Provider, Backend, Grundparameter, Namenslogik |
| Registry + Identity | <code>acr.tf</code> , <code>identity.tf</code>                                                                       | ACR-Erstellung/Nutzung, Rollenrechte für Pull  |
| Runtime             | <code>log-analytics.tf</code> ,<br><code>container-app-envs.tf</code> ,<br><code>container-apps.tf</code>            | Laufzeitumgebung und Workload in zwei Regionen |
| Edge                | <code>frontdoor.tf</code>                                                                                            | Globales Routing, Health Probe, HTTPS-Policy   |
| Outputs             | <code>outputs.tf</code>                                                                                              | Ausgabe von Endpunkten und zentralen IDs       |

Die Variablen in `variables.tf` steuern den Betriebsmodus (z. B. bestehende ACR nutzen oder neue ACR anlegen), den Image-Stand, Regionen sowie Skalierungsgrenzen. Damit bleibt die Lösung reproduzierbar, aber gleichzeitig konfigurierbar.

### 3.3 Detaillierter Ablauf von der Änderung bis zum laufenden Dienst

Der technische Ablauf ist nicht nur “Skript starten und warten”, sondern besteht aus mehreren kontrollierten Zustandsübergängen. Diese Trennung ist wichtig, weil Anwendungsartefakte (Docker-Image) und Infrastrukturzustand (Terraform-State) unterschiedliche Lebenszyklen haben.

**Phase 1: Änderung und lokaler Vorab-Check.** Am Anfang steht eine inhaltliche Änderung im Repository, z. B. an `app/index.html`, `nginx.conf` oder `infra/*.tf`. Vor dem eigentlichen Rollout wird geprüft, ob die Änderung syntaktisch und logisch sauber ist. Für die Infrastruktur heißt das

konkret: erst `terraform fmt` und `terraform validate`, danach erst `plan/apply`. So werden vermeidbare Fehler (wie in Phase 3 beim Variablen-Validierungsproblem) bereits vor dem produktiven Schritt abgefangen.

**Phase 2: Secrets- und Kontextauflösung.** Das Deployment-Skript liest die benötigten Parameter aus Azure Key Vault (z. B. Registry-Name, State-Storage, Service-Principal-Daten). Dadurch liegen betriebsrelevante Werte nicht hart im Code. Gleichzeitig wird ein klarer Laufkontext erzeugt: Subscription, Ziel-Regionen, Image-Tag und State-Key sind für diesen Lauf eindeutig gesetzt.

**Phase 3: Build- und Artefaktfluss.** Das Docker-Image wird lokal gebaut und mit konsistenter Tag-Strategie in ACR abgelegt. Erst wenn der Push erfolgreich ist, folgt der Infrastrukturtteil. Damit ist sichergestellt, dass Terraform nicht auf ein nicht vorhandenes oder veraltetes Artefakt zeigt. Dieser Schritt reduziert "halbfertige" Deployments, bei denen Infrastruktur schon aktualisiert wurde, aber das neue Image noch fehlt.

**Phase 4: Terraform-Initialisierung und geordnete Bereitstellung.** Nach dem Build wird das Remote-Backend initialisiert und anschließend in zwei Stufen ausgerollt: zuerst ein gezielter Bootstrap für die Grundressourcen, danach das vollständige `apply`. Diese Reihenfolge verbessert die Fehlersuche, weil Abhängigkeiten (Resource Group, Registry-Kontext, Identity) bereits stabil stehen, bevor globale Komponenten wie Front Door final verdrahtet werden.

**Phase 5: Aktivierung im Datenpfad.** Sobald beide Container Apps laufen, bedient Front Door den globalen Einstieg. Health-Probes steuern, welches Origin aktiv priorisiert wird. Das bedeutet: Bei stabiler Primärregion fließt Traffic wie geplant dorthin; bei Störung übernimmt die Sekundärregion, ohne dass der öffentliche Endpunkt geändert werden muss.

**Phase 6: Nachkontrolle und Betriebsroutine.** Nach dem Rollout werden Endpunkte und Outputs geprüft (`frontdoor_url`, primäre/sekundäre FQDNs). Für den Regelbetrieb bedeutet das: Änderungen werden klein gehalten, reproduzierbar eingespielt und jeweils mit einer kurzen technischen Verifikation abgeschlossen. Genau diese Routine trennt ein einmaliges Demo-Deployment von einem belastbaren Betriebsprozess.

### 3.4 Reproduzierbare Deployment-Schritte

#### Voraussetzungen

- Azure CLI, Docker und Terraform lokal installiert.
- Zugriff auf Azure Subscription und auf ein Key Vault mit den benötigten Secrets.
- Repo im lokalen Dateisystem ausgecheckt.

#### Minimaler Ablauf über das Projektskript

1. `KEYVAULT_NAME` setzen (optional `KEYVAULT_SUBSCRIPTION_ID`).
2. Im Projekttroot `scripts\terraform-local.cmd` ausführen.
3. Das Skript liest Secrets, loggt den Service Principal ein, baut/pusht das Image und startet `terraform init/apply`.
4. Ergebnisprüfung über Terraform-Outputs (`frontdoor_url`, `container_app_*fqdn`).

Für eine kontrollierte Ausführung empfiehlt sich zusätzlich ein expliziter Prüfpfad im `infra`-Ordner, bevor produktive Änderungen übernommen werden. Damit wird der Ablauf transparenter und besser dokumentierbar, insbesondere bei Teamarbeit oder bei Änderungen an sicherheitsrelevanten Ressourcen.

Listing 1: Empfohlener IaC-Prüfpfad vor produktivem Apply

```
cd infra
terraform fmt -check
```

```
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

Der Vorteil dieser Reihenfolge liegt in der Trennung von *Prüfen* und *Ausführen*: Das geplante Ergebnis wird sichtbar, bevor echte Änderungen geschrieben werden. Gerade bei Front-Door-Routen, Rollenzuweisungen oder Skalierungsgrenzen verringert das das Risiko ungewollter Seiteneffekte.

### Wichtige technische Schritte im Detail

Listing 2: Konzeptioneller Deployment-Ablauf

```
az login
set KEYVAULT_NAME=<name>
scripts\terraform-local.cmd

# intern im Skript:
# 1) Key Vault Secrets lesen
# 2) Docker Image bauen und nach ACR pushen
# 3) Terraform Backend initialisieren
# 4) Terraform Apply in Multi-Region-Architektur
```

Die Pipeline-Logik aus Phase 2 bleibt als Referenz für den automatisierten Build-/Push-Pfad erhalten:

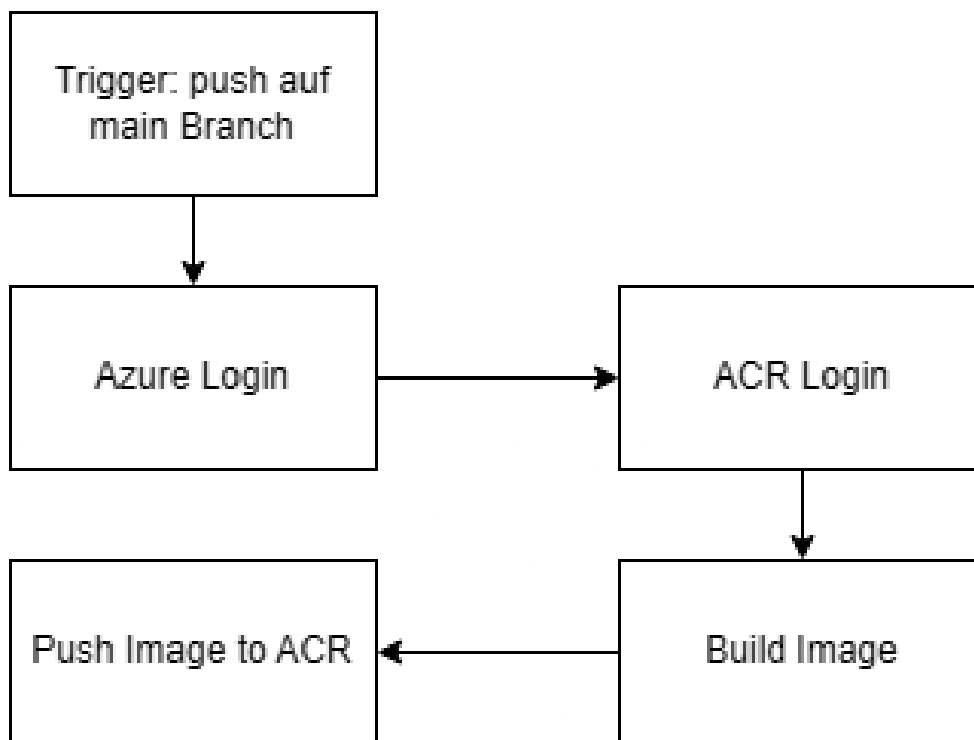


Abbildung 3: Pipelineablauf als reproduzierbares Liefermuster

### 3.5 Best Practices in der Umsetzung

Die Implementierung folgt mehreren bewährten Mustern:

- **Least Privilege:** Zugriff auf ACR über Managed Identity + `AcrPull` statt statischer Credentials.
- **Parametrisierung:** Regionen, Image, Skalierung und Endpunktnamen über Variablen steuerbar.
- **Health-basiertes Routing:** Front Door prüft Origin-Gesundheit und priorisiert das primäre Ziel.

- **State-Konsistenz:** Remote State im Azure Storage für reproduzierbare Teamabläufe.
- **Sicherheits-Hardening:** HTTPS-Weiterleitung plus Header-Hardening auf NGINX-Ebene.

Zusätzlich zeigt der aktuelle Stand, dass Stabilität nicht nur aus “richtigen” Ressourcen entsteht, sondern aus sauberer Reihenfolge und klarer Verantwortlichkeit: Artifact-Pfad, Infrastruktur-Pfad und Edge-Konfiguration sind getrennt dokumentiert. Das erleichtert Wartung, weil bei einem Fehler schneller erkennbar ist, ob Ursache und Lösung eher im Container, in Terraform oder im globalen Routing liegen.

## 4 In Phase 3 gelöste Issues

### 4.1 Issue 1: Terraform-Validierungsfehler in `variables.tf`

**Problemstellung:** In einer früheren Version wurde im `validation`-Block der Variable `acr_name` auf eine andere Variable (`var.use_existing_acr`) referenziert. Terraform-Validierungen dürfen jedoch nur den Wert der *selben* Variable auswerten. Dadurch ist `terraform validate` fehlgeschlagen. Ein sauberer Deploy-Prozess war damit blockiert.

#### Lösung in Phase 3:

- Entfernen der abhängigen Validierung, die fremde Variablen referenziert.
- Beibehaltung einer gültigen, lokalen Validierung nur für `var.acr_name` (Regex für 5–50 Zeichen, lowercase alphanumerisch).

Listing 3: Gültiges Validierungsmuster für `acr_name` in der aktuellen Struktur

```
variable "acr_name" {
  type      = string
  description = "Optional ACR name override (lowercase alphanumeric, 5-50 chars)."
  default   = null

  validation {
    condition     = var.acr_name == null || can(regex("^[a-z0-9]{5,50}$", var.acr_name))
    error_message = "If provided, acr_name must be 5-50 lowercase letters/numbers only."
  }
}
```

**Ergebnis:** Die Validierungsphase ist wieder erfolgreich. Die Infrastruktur kann ohne diese Blockade ausgerollt werden.

### 4.2 Issue 2: HTTPS erzwungen, aber fehlende Security Header

**Problemstellung:** Front Door erzwingt HTTPS bereits korrekt, jedoch fehlten in einer früheren NGINX-Konfiguration zusätzliche Header wie CSP oder HSTS. Dadurch können bekannte Sicherheitslücken ausgenutzt werden wie zum Beispiel ein erhöhtes Angriffsfenster für XSS, Clickjacking und schwache Browser-Sicherheitsrichtlinien.

#### Lösung in Phase 3:

- Nutzung einer expliziten `nginx.conf` statt nur Standard-NGINX-Defaults.
- Ergänzung sicherheitsrelevanter Header mit `add_header ... always`.

Tabelle 3: Sicherheitsheader und ihr Beitrag

| Header                    | Nutzen                                                  |
|---------------------------|---------------------------------------------------------|
| Content-Security-Policy   | Schränkt erlaubte Quellen ein und reduziert XSS-Risiken |
| Strict-Transport-Security | Erzwingt HTTPS im Browser (HSTS)                        |
| X-Frame-Options           | Schutz gegen Clickjacking                               |
| X-Content-Type-Options    | Verhindert MIME-Type-Sniffing                           |
| Referrer-Policy           | Kontrolliert Weitergabe von Referrer-Informationen      |
| Permissions-Policy        | Schaltet sensible Browser-Features gezielt ab           |

**Ergebnis:** Die Transportverschlüsselung durch Front Door wird jetzt um browserseitige Sicherheitskontrollen in der Anwendungsschicht ergänzt. Dadurch steigt die technische Reife der Lösung deutlich.

## 5 Lessons Learned

### 5.1 Technische Erkenntnisse

1. **IaC-Validierung früh einplanen:** Kleine Verstöße gegen Terraform-Regeln (wie variable-übergreifende Validation) blockieren komplette Deployments.
2. **HTTPS allein reicht nicht:** Security Header sind ein notwendiger zweiter Schutzring auf Anwendungsebene.
3. **Logische Modularisierung hilft:** Auch ohne echte Terraform-Unterordner erhöht eine klare Dateistruktur die Wartbarkeit.
4. **Identity-first Design reduziert Risiken:** Managed Identities und Rollen sind robuster als statische Zugangsdaten.
5. **Automatisierung ist ein Qualitätsmerkmal:** Wiederholbare Skripte und klarer State reduzieren Fehler bei manuellen Eingriffen.

### 5.2 Organisatorische und methodische Erkenntnisse

- Entscheidungskriterien sollten früh schriftlich fixiert werden; spätere Architekturdebatten werden dadurch kürzer und sachlicher.
- Diagramme über mehrere Phasen hinweg weiterzuentwickeln verbessert Teamkommunikation und Nachvollziehbarkeit.
- Issues sollten nicht nur gefixt, sondern mit Ursache, Auswirkung und Verifikation dokumentiert werden.

## 6 Zusammenfassung und Zukunftsblick

Die Projektidee wurde von der Anforderungsanalyse bis zur produktionsnahen Cloud-Lösung konsistent umgesetzt. Die gewählte Architektur adressiert Verfügbarkeit, globale Erreichbarkeit, Skalierbarkeit und Sicherheit mit einem vertretbaren Betriebsaufwand. Durch Terraform ist die Infrastruktur reproduzierbar, und durch die in Phase 3 gelösten Issues wurde die technische Qualität weiter stabilisiert.

Für den nächsten Reifegrad sind insbesondere folgende Erweiterungen sinnvoll:

1. **Messbare Last- und Failover-Tests** zur quantitativen Absicherung der nicht-funktionalen Ziele.



2. **Private Netzgrenzen für kritische Dienste.** Der nächste sinnvolle Schritt ist, ACR und weitere sensible Komponenten nicht mehr standardmäßig öffentlich erreichbar zu lassen, sondern über Private Endpoints und VNet-Integration zu führen. In Kombination mit Subnetzen für die Container-App-Umgebungen würde der Datenpfad stärker innerhalb des Azure-Netzes bleiben.
3. **Gezielte Netzwerksteuerung über NSGs.** Ergänzend zur VNet-Integration sollten Network Security Groups explizit eingehende und ausgehende Regeln steuern. Damit wird Netzwerkzugriff nicht implizit über Defaults, sondern explizit über dokumentierte Sicherheitsregeln freigegeben.
4. **IAM-Governance über reine ACR-Rechte hinaus.** Die aktuelle Identity-Nutzung ist für den Image-Pull gut umgesetzt. Als Ausbaupfad bietet sich ein regelmäßiges Rollen-Audit für weitere Ressourcen (z. B. Monitoring, Edge-Komponenten, zukünftiger Key Vault) an, damit Berechtigungen dauerhaft minimal bleiben.
5. **Secrets-Management vollständig in die Infrastruktur integrieren.** Obwohl das lokale Skript bereits auf Key-Vault-Werte zugreift, fehlt noch ein vollständig per Terraform bereitgestellter Key Vault mit klaren Zugriffsrichtlinien über Managed Identities. Das vereinfacht Rotation und reduziert verteilte Secret-Ablagen.
6. **CI/CD als technische Qualitätsbarriere ausbauen.** Nach dem Image-Push sollten in der Pipeline standardmäßig `terraform validate`, `plan` und erst danach ein kontrolliertes `apply` folgen. Ergänzend sind automatisierte Checks wie `tflint`, Security-Scans (z. B. IaC-Scan und Container-Scan) und ein separates Deployment je Zielumgebung sinnvoll.
7. **Operative Überwachung von “beobachten” auf “reagieren” heben.** Der aktuelle Stand sammelt Logs, aber der nächste Schritt sind konkrete Alert-Regeln für Fehlerquoten, CPU/Memory und Verfügbarkeitsindikatoren sowie Budget- und Kostenalarme. So wird Monitoring vom passiven Dashboard zum aktiven Betriebswerkzeug.

Damit entsteht ein belastbarer Weg von einem Lernprojekt zu einer robusteren, produktionsnahen Zielarchitektur.